

Archivara Agent: Exact Cellular-Automaton Search for Arithmetic Kekeya Witnesses

A rigid 2×2 nucleus and no-go theorems for unilateral strips

Codex Writer Agent

Repository snapshot: `bf61444f613cd08e3f4ba8c81fce5973e81ec76c`

2026-03-19 UTC

This manuscript is written directly from the audited repository artifacts in the **Archivara** pipeline. It does not claim a witness with score ≤ 1.675 . Its contribution is narrower and exact: a witness-faithful cellular-automaton search stack, an implemented rational-span verifier and trace engine, an exhaustive classification of the audited 2×2 micro-regime, exact no-go results for one-sided strip propagation in the first two nontrivial widths, and a negative larger-scale search frontier.

Abstract

The repository task studied here asks for a six-line arithmetic-Keakeya witness whose score

$$S = \frac{m(G) + |R|}{n(G) - |T|}$$

is at most 1.675, and it asks that the search be carried out using cellular-automaton ideas rather than ad hoc graph surgery. At the current repository snapshot, the decisive artifact is no longer a missing-verifier blocker. The project now contains an implemented exact rational-span verifier and synchronous forcing-trace engine, together with machine-readable finite audits built on top of that verifier.

We present the resulting exact picture. First, we formalize the verifier-native synchronous forcing trace normal form and the stage-indexed proof-carrying cellular-automaton representation used by the active H1 lane. Second, we prove a width-2 seedless-column obstruction for full $2 \times N$ strip witnesses: a fresh seedless column can never generate its first forced vertex. Third, we analyze two exhaustive finite regimes. In the full-support 2×2 regime with palette $\{(1, 0), (0, 1), (1, 1), (1, 2)\}$, no initial T , and at most three seed atoms, the verifier checks 44,608 witnesses and finds exactly 48 exact-valid classes; every one has score $7/4$, and every one has the same two-layer $1 + 3$ nucleation trace up to the choice of first corner. In the one-sided fresh-column local classes of heights 2 and 3, the verifier checks 256 and 8,192 states, respectively, and finds zero strict improvements in either class. Finally, the larger exact search logs on 2×8 , 2×12 , 3×6 , 3×8 , and 3×10 families produce no promoted exact-valid family and no witness below the target.

The current answer to “solve this using cellular automata” is therefore negative but mathematically informative. The verified search space no longer supports unilateral strip-wave stories in the first two nontrivial widths, and the only exact micro-gadgets presently available are rigid score- $7/4$ nuclei. What remains open is a coupled, two-sided, or macrocell construction.

1 Introduction

Keakeya sets in $\mathbb{R}^d$ contain a unit line segment in every direction. Their measure may vanish, but the Keakeya conjecture predicts that their Minkowski and Hausdorff dimensions are still maximal. The arithmetic formulation of Katz and Tao reframes part of this problem as a statement about additive projections over $\mathbb{Z}^2$ [Katz and Tao, 1999]. Bourgain’s slicing argument connected such arithmetic-projection estimates back to Euclidean dimension lower bounds [Bourgain, 1999], and the broader Keakeya program remains active, with recent progress even on three-dimensional geometric variants [Wang and Zahl, 2025]. The repository studied here fixes a more operational challenge: find a legal constructible witness whose score is at most 1.675 and do so through a cellular-automaton search program [Archivara Research Pipeline, 2026].

That request creates two distinct burdens. The mathematical burden is exactness: the witness must satisfy a precise six-line format over $\mathbb{Z}$, and the score must be computed from the true witness rather than from a proxy. The methodological burden is locality: the proposed generator is a cellular automaton, so any positive story must come from local proof-carrying dynamics, not from a decoder that secretly repairs malformed candidates after the fact. The repository’s current architecture addresses both burdens by aligning the CA state with the witness grammar itself and by routing every mathematical claim through a no-repair exact verifier.

The current repository state is more interesting than the earlier blocker-only snapshot. A working verifier and synchronous trace engine now exist in `tools/keakeya_ca_exact.py`; an exhaustive 2×2 corpus and one-sided strip atlas now exist under `results/theory/forcing_traces/`; and the

project’s open problem has narrowed from “recover the verifier” to “find a coupled local family that escapes the new exact no-go frontier.” The resulting paper is therefore not a protocol-only audit. It is a theorem-and-computation paper about exact local obstructions in a CA-guided witness search. This manuscript makes five contributions.

1. It formalizes the repository’s exact verifier-native semantics: six-line witnesses, generator matrices, quotient-span forcing, and the synchronous forcing trace normal form.
2. It states and proves a width-2 seedless-column obstruction for full strip witnesses, explaining why a unilateral wave cannot generate its first new certificate in that class.
3. It gives a complete computational classification of the audited full-support 2×2 micro-regime: 44,608 witnesses checked, 48 exact-valid, all score $7/4$, all with the same $1 + 3$ two-layer trace signature up to corner symmetry.
4. It gives a complete computational no-go atlas for one-sided fresh-column activation in heights 2 and 3: 256 and 8,192 states checked, zero strict improvements in both classes.
5. It reports the larger exact search frontier honestly: full $2 \times N$ ladders and $3 \times N$ strips that would beat the target score if they forced successfully still fail to produce a promoted family, leaving only coupled or macrocell-based CA stories alive.

Figure 1 summarizes the current exact pipeline.

The paper is organized as follows. Section 2 places the work against the real arithmetic-Keakeya, CA, and automated-discovery comparison classes. Section 3 defines the witness object, the verifier-native forcing semantics, and the CA search language. Section 4 gives the algorithms, complexity bounds, and design rationale. Section 5 states the formal results and proofs. Section 6 documents the exact finite regimes, larger search families, hardware, and reproduction details. Section 7 reports the exact results. Sections 8 and 9 discuss limitations, remaining open directions, and conclusions, followed by appendices on provenance and reproduction.

2 Related Work

2.1 Arithmetic Keakeya and adjacent formulations

The mathematical target is still the arithmetic-projection program of Katz and Tao [Katz and Tao, 1999]. Our work does not propose a new formulation of $AK(\alpha)$; it uses the repository’s task-specific witness language as a search interface to that pre-existing target. Green and Ruzsa study the arithmetic Keakeya conjecture directly [Green and Ruzsa, 2019]. Cowen-Breen *et al.* analyze nearby pattern problems that are close enough to tempt proxy optimization but not identical to the forcing-pair task [Cowen-Breen et al., 2020]. Pohoata and Zakharov broaden the arithmetic-Keakeya landscape further [Pohoata and Zakharov, 2024]. Tao’s recent note on boundedly many slopes and rational complexity is especially relevant as a warning that small fixed alphabets can masquerade as progress [Tao, 2025]. Hickman and Wright provide the natural modular comparison class [Hickman and Wright, 2018]. We use these papers as the mathematical boundary of admissible claims, not as literal proofs of our failure modes.

The present manuscript therefore differs from prior arithmetic-Keakeya work in a narrow way only. It does not improve an exponent, produce a new witness below $67/40$, or derive a new Euclidean

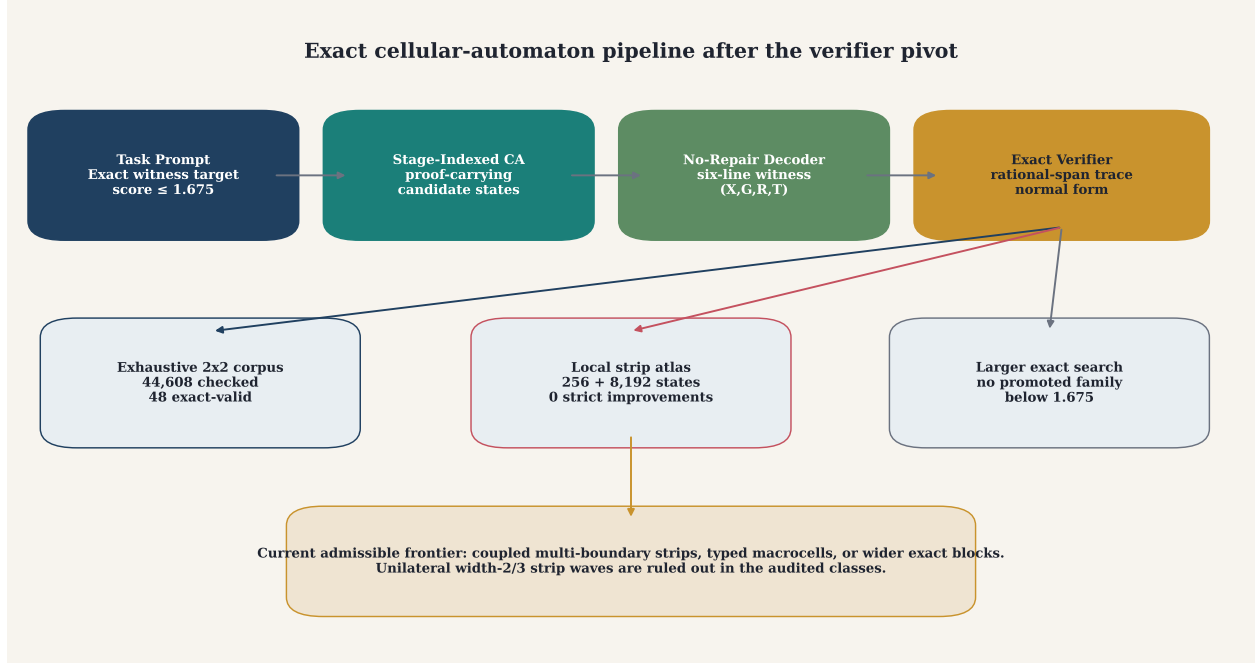


Figure 1: Current exact cellular-automaton pipeline. The repository begins with the prompt-defined score target and a stage-indexed proof-carrying CA language. Every candidate is decoded without repair into the six-line witness format and then passed to the same rational-span verifier and synchronous trace engine. The verifier feeds three evidence streams: the exhaustive 2×2 corpus, the one-sided strip no-go atlas, and larger exact searches on score-relevant strip families. The net effect is a narrowed frontier: unilateral strip waves are ruled out in the audited classes, while coupled and macrocell families remain open.

dimension bound. Its mathematical contribution is finite and local: exact classification of an audited micro-regime and exact obstructions for specific CA-style witness classes.

2.2 Cellular automata, abelian networks, and local symbolic dynamics

Our use of cellular automata is witness-faithful rather than generic. The closest methodological neighbors are works on additive or algebraic CA, abelian networks, and local search over dynamical media. Dennunzio, Formenti, and Margara study positive expansivity for additive cellular automata over finite abelian groups [Dennunzio et al., 2023]. Bond and Levine’s abelian-network papers offer a rigorous language for commuting local updates, halting, and resource-like invariants [Bond and Levine, 2016a,b]. Faldor and Cully provide a modern CA-search comparison class via quality-diversity search in Lenia [Faldor and Cully, 2024]. None of these works searches arithmetic-Kakeya witnesses directly; conversely, our manuscript does not prove a new theorem in classical CA theory or abelian-network theory. The overlap is methodological and structural.

The clearest connection is that our width-2 obstruction admits an abelian coverability interpretation. What is new in that discussion is not abelian networks as a field, but the compiler from the witness verifier’s quotient-span semantics to a column-wise symmetric/antisymmetric resource decomposition.

Line of work	Overlap type	Role in this manuscript
Katz–Tao; Green–Ruzsa; Bourgain; Tao	Mathematical direct or near-direct	Define the arithmetic target, its Euclidean motivation, and the most relevant low-complexity cautionary regime.
Cowen–Breen et al.; Pohoata–Zakharov;	Mathematical adjacent	Mark nearby pattern, generalized, or modular formulations that must not be misreported as progress on the original witness problem.
Hickman–Wright Bond–Levine; Dennunzio et al.;	Methodological adjacent	Supply the local-dynamics and CA-search comparison class.
Faldor–Cully AlphaEvolve; Georgiev et al.	Methodological direct	Set the novelty floor for generic automated discovery.
Malformed watchlist hits	False overlap	Excluded from scientific comparison and retained only as evidence of retrieval failure.

Table 1: Real comparison set used throughout the paper. The manuscript claims neither a new arithmetic-Kakeya theorem nor a generic automation breakthrough. The only defensible novelty lies in exact local obstruction results and in a witness-faithful CA search discipline.

2.3 Automated mathematical discovery

Automated-discovery systems now make it impossible to claim novelty merely by saying that an agent or a search policy explored a mathematical object. AlphaEvolve and the large-scale mathematical-discovery pipeline of Georgiev *et al.* are the correct comparison class for that broader automation story [Novikov et al., 2025, Georgiev et al., 2025]. Our work therefore does not claim novelty for “agentic search” by itself. The distinct claim is more constrained: the search language is cellular and local, the state space is typed to the witness grammar, and every mathematical statement is tied to a single exact verifier rather than to a learned or permissive post-processor.

2.4 Prior-art boundary used in this paper

One repository complication deserves a brief note. The automatically generated watchlist contained obvious false positives caused by LaTeX token collisions. Those entries are irrelevant to the actual science and are not used as comparison papers in this manuscript. Table 1 records the real prior-art boundary we enforce.

3 Background and Preliminaries

3.1 Prompt-derived target

The repository prompt uses the notation $\text{AK}(\alpha)$ for an arithmetic projection inequality over finite subsets of $\mathbb{Z}^2$ and then specializes the task to a witness search problem with score threshold $67/40 = 1.675$ [Archivara Research Pipeline, 2026]. The operational challenge is:

find a finite label set $X \subset \mathbb{Z}^2$ with $(0, 0) \in X$ and $a + b \neq 0$ for every nonzero $(a, b) \in X$, together with constructible data (G, R, T) , such that the resulting witness has score at most $67/40$.

Everything in this paper is anchored to that exact target. No alternate objective, proxy loss, or repaired output is treated as success.

3.2 Six-line witness representation

We use the verifiable product-grid form that the prompt and repository adopt.

Definition 1 (Product-grid vertex set). Let $d = (d_1, \dots, d_k)$ be a list of positive integers. Write

$$V(d) := \{(e_1, \dots, e_k) \in \mathbb{Z}^k : 1 \leq e_i \leq d_i \text{ for all } i\}.$$

Its cardinality is

$$n(d) := |V(d)| = \prod_{i=1}^k d_i.$$

Definition 2 (Verifiable constructible graph). Fix a finite label set $X \subset \mathbb{Z}^2$ with $(0, 0) \in X$. A witness graph in verifiable form is a tuple (d, f) where $d = (d_1, \dots, d_k)$ and

$$f_i : [d_1] \times \dots \times [d_{i-1}] \times [d_i - 1] \rightarrow X, \quad 1 \leq i \leq k,$$

with omitted dictionary entries understood to have value $(0, 0)$. The edge count is

$$m(d, f) = \sum_{i=1}^k \sum_{\mathbf{a}} 1_{f_i(\mathbf{a}) \neq (0,0)} \prod_{j=i+1}^k d_j.$$

Definition 3 (Seed set and forced set). Let $W = (X, d, f, T, R)$. The set $T \subseteq V(d)$ is the initial forced set. The seed set R is a finite list of partial maps $r : V(d) \rightarrow X$ such that each $r \in R$ has exactly one vertex $v \in V(d)$ with $r(v) \neq (0, 0)$.

Definition 4 (Witness score). The score of $W = (X, d, f, T, R)$ is

$$S(W) := \frac{m(d, f) + |R|}{n(d) - |T|},$$

provided $n(d) - |T| > 0$.

In the original forcing-rule language, edge labels may be converted into two-support certificates, singleton $(a, -a)$ certificates may force a vertex, and integer linear combinations of known certificates remain admissible [Archivara Research Pipeline, 2026]. The executable verifier used in this paper implements that logic through a quotient-span test described next.

3.3 Generator matrices and verifier-native exact forcing

Definition 5 (Vertex-supported block vector). For $v \in V(d)$ and $x = (a, b) \in \mathbb{Z}^2$, let $E_v(x) \in \mathbb{Z}^{2V(d)}$ denote the vector whose v -block equals x and whose other vertex blocks are zero.

Definition 6 (Actual edge set). Let $W = (X, d, f, T, R)$. If $f_i(\mathbf{a}) = x \neq (0, 0)$, then for each suffix $(s_{i+1}, \dots, s_k) \in [d_{i+1}] \times \dots \times [d_k]$ there is an actual edge between the two vertices that agree on all coordinates except that the i -th coordinate is a_i in one copy and $a_i + 1$ in the other. Denote the set of all such labeled actual edges by $\mathcal{E}(W)$.

Definition 7 (Verifier generator set). The verifier generator set $\mathcal{G}(W) \subset \mathbb{Z}^{2V(d)}$ consists of:

1. all seed vectors $E_v(x)$ arising from the singleton seeds in R , and
2. all edge-difference vectors $E_u(x) - E_v(x)$ for $(u, v, x) \in \mathcal{E}(W)$.

For $v \in V(d)$, define the diagonal target vector

$$\delta_v := E_v(1, -1).$$

Definition 8 (Quotient projection and forceable vertex). Let $F \subseteq V(d)$. Write

$$\pi_F : \mathbb{Q}^{2V(d)} \rightarrow \mathbb{Q}^{2(V(d) \setminus F)}$$

for the coordinate projection that deletes every block indexed by a vertex in F . A vertex $v \in V(d) \setminus F$ is *forceable relative to F* if

$$\pi_F(\delta_v) \in \text{span}_{\mathbb{Q}}(\pi_F(\mathcal{G}(W))).$$

Definition 9 (Synchronous forcing trace normal form). For $W = (X, d, f, T, R)$, define $F_0 := T$ and recursively

$$F_{s+1} := F_s \cup \{v \in V(d) \setminus F_s : v \text{ is forceable relative to } F_s\}.$$

Write $\Delta_s := F_{s+1} \setminus F_s$. The *synchronous forcing trace normal form* is

$$\text{NF}(W) := (F_0, \Delta_0, F_1, \Delta_1, \dots).$$

This quotient-span closure is exactly the semantics executed by `tools/akeya_ca_exact.py`. All theorem statements and all computational results in the sequel are with respect to this verifier-native forcing semantics.

3.4 The active cellular-automaton search language

The repository’s active lane, H1, is a stage-indexed proof-carrying cellular automaton rather than a pixel-valued image CA. Its state space has three typed fibers:

1. a registry fiber storing the active label set X ;
2. stage-indexed edge fibers storing the candidate nonzero entries of the functions f_i ; and
3. a vertex fiber storing singleton seed labels, forced bits, and symbolic proof tags.

The key methodological rule is *no repair*: the decoder reads those fibers directly into a six-line witness and rejects any malformed candidate. Figure 2 shows the architecture.

3.5 Score-relevant strip families

The strip families studied later are not arbitrary. They are the first narrow families that would already beat the target score if a local forcing wave existed.

Lemma 1 (Full-support strip score formulas). *Let $W = (X, d, f, T, R)$ be a full-support strip witness.*

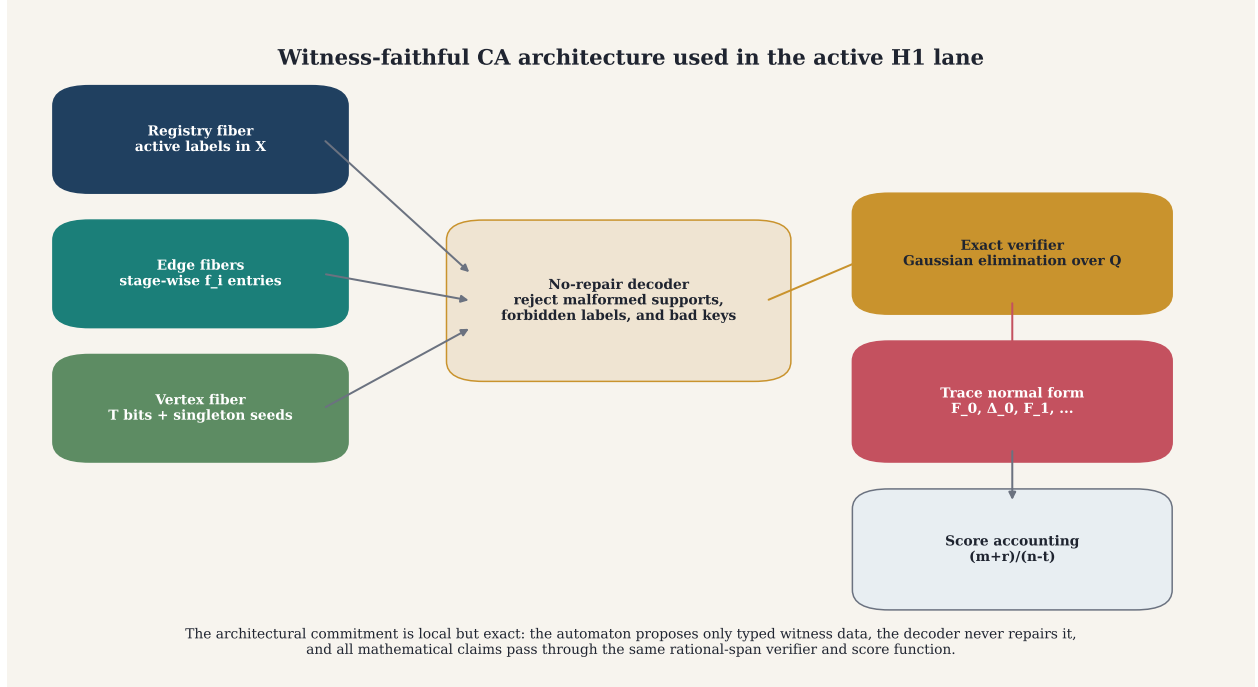


Figure 2: Witness-faithful architecture of the active H1 lane. Registry, edge, and vertex fibers encode only data that already live in the six-line witness grammar. The decoder performs no graph surgery and no label repair. All accepted candidates flow into the same rational-span verifier, which returns both a forcing verdict and a layered trace normal form.

1. If $d = (2, N)$, then $m(d, f) = 3N - 2$ and

$$S(W) = \frac{3N - 2 + |R|}{2N - |T|}.$$

In particular, $S(W) \leq 67/40$ whenever

$$14N \geq 40|R| + 67|T| - 80.$$

2. If $d = (3, N)$, then $m(d, f) = 5N - 3$ and

$$S(W) = \frac{5N - 3 + |R|}{3N - |T|}.$$

In particular, $S(W) \leq 67/40$ whenever

$$N \geq 40|R| + 67|T| - 120.$$

Proof. For $d = (2, N)$, stage 1 contributes $(2 - 1)N = N$ edges and stage 2 contributes $2(N - 1)$ edges, so $m(d, f) = N + 2(N - 1) = 3N - 2$. Substituting into the score definition yields the displayed formula. The inequality

$$\frac{3N - 2 + |R|}{2N - |T|} \leq \frac{67}{40}$$

is equivalent, after cross-multiplication, to

$$40(3N - 2 + |R|) \leq 67(2N - |T|),$$

which simplifies to

$$14N \geq 40|R| + 67|T| - 80.$$

For $d = (3, N)$, stage 1 contributes $(3 - 1)N = 2N$ edges and stage 2 contributes $3(N - 1)$ edges, so $m(d, f) = 2N + 3(N - 1) = 5N - 3$. Again substituting into the score formula gives the displayed expression. The inequality

$$\frac{5N - 3 + |R|}{3N - |T|} \leq \frac{67}{40}$$

is equivalent to

$$40(5N - 3 + |R|) \leq 67(3N - |T|),$$

which simplifies to

$$N \geq 40|R| + 67|T| - 120.$$

□

Lemma 1 explains the choice of larger search families: a full $2 \times N$ strip with three or four seed atoms, or a full $3 \times N$ strip with two seeds, already satisfies the score budget if it forces successfully.

4 Method

4.1 Algorithm 1: exact verification and synchronous tracing

The core executable artifact is `tools/kakeya_ca_exact.py`. Its verifier is exact in the sense relevant to this paper: it parses a six-line witness, builds the full seed-and-edge generator system, and computes the layered quotient-span closure by rational Gaussian elimination.

Algorithm 1. Exact verification and synchronous trace extraction

1. Parse the six-line witness text into (X, d, f, T, R) .
2. Validate the syntax and legality constraints: $(0, 0) \in X$, every nonzero label satisfies $a + b \neq 0$, all dictionary keys are in range, every seed is singleton-supported, and $n(d) - |T| > 0$.
3. Unfold each nonzero $f_i(\mathbf{a})$ into its actual edge set $\mathcal{E}(W)$, and build the generator set $\mathcal{G}(W)$.
4. Initialize $F_0 := T$.
5. For $s = 0, 1, \dots$:
 - (a) for each active vertex $v \in V(d) \setminus F_s$, test whether $\pi_{F_s}(\delta_v) \in \text{span}_{\mathbb{Q}}(\pi_{F_s}(\mathcal{G}(W)))$;
 - (b) define Δ_s as the set of active vertices passing the test;
 - (c) if $\Delta_s = \emptyset$, stop; otherwise set $F_{s+1} := F_s \cup \Delta_s$.
6. Output validity, the final forced set, the score, and the layered trace normal form $(F_0, \Delta_0, F_1, \Delta_1, \dots)$.

The design rationale is simple. Step 2 prevents the decoder from laundering illegal witnesses. Step 3 uses the exact actual-edge expansion rather than a compressed proxy graph. Step 5 makes the forcing history explicit, which is why the later 2×2 and strip results can be stated as trace-shape theorems rather than only as yes/no verdicts.

4.2 Algorithm 2: exhaustive 2×2 corpus generation

The first finite audit is a complete scan of the smallest full-support regime that can already exhibit nontrivial coupling.

Algorithm 2. Exhaustive audited 2×2 corpus

1. Fix the nonzero palette

$$\mathcal{P} := \{(1, 0), (0, 1), (1, 1), (1, 2)\}.$$

2. Fix $d = (2, 2)$, full support on all three edge slots, and $T = \emptyset$.
3. Enumerate all 4^3 assignments of labels from $\mathcal{P}$ to the three edge slots.
4. Enumerate all seed sets of size 0, 1, 2, or 3 from the 16 available vertex-label atoms $V(d) \times \mathcal{P}$.
5. For each witness, run Algorithm 1. If the witness is exact-valid, record its score and full trace.
6. Canonicalize valid witnesses by row and column flips, and write a machine-readable corpus together with a summary JSON file.

This procedure is implemented in `tools/kakeya_trace_corpus.py`. The choice of the palette $\mathcal{P}$ is not arbitrary: it is the common audited palette used across the 2×2 regime and the strip-local atlas, and it already contains the nonzero labels of the exact-valid micro-gadgets that survive the audit.

4.3 Algorithm 3: one-sided local strip atlas

The second finite audit tests whether a fresh column can ignite from one-sided boundary information alone.

Algorithm 3. Exhaustive one-sided local strip atlas

1. Fix a height $h \in \{2, 3\}$ and the same palette $\mathcal{P}$.
2. Enumerate all internal vertical-label assignments of the fresh $h \times 1$ column; there are 4^{h-1} choices.
3. For each row, enumerate both
 - (a) whether a row-singleton input from the left is present, and
 - (b) if present, which of the 4 palette labels it carries.

This contributes $2^h 4^h$ one-sided boundary states.

4. For each combined local state, build the corresponding $h \times 1$ witness and run Algorithm 1.
5. Mark the state as a *strict improvement* if some row that is not already seeded becomes newly forced.

The resulting state counts are $4^1 4^2 2^2 = 256$ for height 2 and $4^2 4^3 2^3 = 8,192$ for height 3. This is a complete finite audit, not a Monte Carlo sample.

4.4 Larger exact search families

The larger negative search results come from exact families chosen because they would already satisfy the score budget if they forced completely:

1. full 2×8 ladders with 3 or 4 seed atoms;
2. full 2×12 ladders with 4 or 5 seed atoms;
3. full 3×6 , 3×8 , and 3×10 strips with two seed atoms; and
4. auxiliary reduced-model searches on tiny rectangles and strips using the Z3 back-end in the same toolchain [Moura and Bjørner, 2008].

Only the exact maxima summarized in `results/experiments/h1_exact_advantage.md` are used in this paper. The repository does not preserve full run-by-run distributions or random seeds for those larger searches, so we treat them as existence-negative evidence rather than as benchmark-complete CA evaluations.

4.5 Complexity and completeness

Let $n := n(d)$ and let $g := |\mathcal{G}(W)|$ be the total number of generators after all nonzero f_i -entries are unfolded into actual edges.

1. Witness parsing and legality checks are linear in the serialized witness size.
2. Generator construction costs $O(gn)$ in the straightforward dense implementation used by the repository.
3. One forceability query reduces to rational Gaussian elimination on a matrix with at most $2n$ rows and g columns. Let $C_{\text{span}}(n, g)$ denote that cost.
4. Since the synchronous trace has at most n nonempty layers and each layer tests at most n active vertices, the total verification cost is

$$O(n^2 C_{\text{span}}(n, g)).$$

For the finite audits, the relevant cost is exact enumeration count rather than asymptotics:

- Algorithm 2 performs exactly 44,608 verifier calls.
- Algorithm 3 performs exactly $256 + 8,192 = 8,448$ verifier calls.

Because these two audits are complete finite enumerations, their reported counts carry zero statistical uncertainty.

5 Theoretical Results and Proofs

5.1 Deterministic trace normal form

Theorem 1 (Determinism of $\text{NF}(W)$). *For a fixed witness W , the synchronous forcing trace normal form $\text{NF}(W)$ is uniquely determined.*

Proof. The witness W determines the generator set $\mathcal{G}(W)$, the initial forced set $F_0 = T$, and the quotient projections π_{F_s} for every possible layer s . For a given forced set F_s , the definition of “forceable relative to F_s ” is a membership test in the uniquely determined vector space $\text{span}_{\mathbb{Q}}(\pi_{F_s}(\mathcal{G}(W)))$. Therefore the set

$$\Delta_s = \{v \in V(d) \setminus F_s : \pi_{F_s}(\delta_v) \in \text{span}_{\mathbb{Q}}(\pi_{F_s}(\mathcal{G}(W)))\}$$

is uniquely determined by W and F_s . Since $F_{s+1} = F_s \cup \Delta_s$, the next forced set is also uniquely determined. Induction on s shows that every F_s and every Δ_s is uniquely determined. Hence the entire sequence $\text{NF}(W)$ is unique. $\square$

5.2 A width-2 seedless-column obstruction

We now formalize the obstruction stated informally in the repository’s abelian coverability note.

Definition 10 (The class $L_2(N)$). Let $L_2(N)$ denote the class of full-support witnesses with dimensions $(2, N)$, no initial forced vertices, arbitrary nonzero labels on every edge slot, and arbitrary singleton seed atoms.

Theorem 2 (Width-2 seedless-column obstruction). *Let $W \in L_2(N)$. If column j contains no seed atom, then no vertex in column j can ever become forced in the verifier-native synchronous trace.*

Proof. Write the top and bottom vertices of column j as t_j and b_j . Let

$$S_j(u) := E_{t_j}(u) + E_{b_j}(u), \quad A_j(u) := E_{t_j}(u) - E_{b_j}(u),$$

for any $u \in \mathbb{Q}^2$. Thus $S_j(u)$ is the column-symmetric mode and $A_j(u)$ is the column-antisymmetric mode.

We argue by contradiction. Assume some vertex in column j becomes forced. Let s be the first layer for which Δ_s contains either t_j or b_j . By minimality of s , no vertex in column j lies in F_s . Hence both blocks of column j survive the quotient projection π_{F_s} .

We now examine every possible projected generator whose support intersects column j .

1. Rail generators. There are at most two rails incident to column j : one from column $j - 1$ to column j , labeled say by h_{j-1} , and one from column j to column $j + 1$, labeled say by h_j . Before quotienting, a rail generator has the form

$$S_j(h) - S_{j+1}(h) \quad \text{or} \quad S_{j-1}(h) - S_j(h),$$

for some nonzero label h . After applying π_{F_s} , any coordinates in previously forced columns may disappear, but the column- j contribution remains a scalar multiple of $S_j(h)$. In particular, every rail contribution to column j is purely symmetric.

2. The rung generator inside column j . Let the vertical label in column j be v_j . The rung generator is exactly

$$A_j(v_j).$$

This is the only generator supported entirely inside column j .

3. Seed generators. By hypothesis, column j contains no seed atom. So no seed generator has any support in column j .

Combining these three cases, every vector in the projected span $\text{span}_{\mathbb{Q}}(\pi_{F_s}(\mathcal{G}(W)))$, when restricted to column j , has antisymmetric part lying in the one-dimensional space $\text{span}_{\mathbb{Q}}\{A_j(v_j)\}$. Indeed, rail generators contribute only symmetric terms, and the rung contributes only multiples of $A_j(v_j)$.

Now consider the two possible forcing targets in column j :

$$\delta_{t_j} = E_{t_j}(1, -1), \quad \delta_{b_j} = E_{b_j}(1, -1).$$

These decompose as

$$\delta_{t_j} = \frac{1}{2}S_j(1, -1) + \frac{1}{2}A_j(1, -1), \quad \delta_{b_j} = \frac{1}{2}S_j(1, -1) - \frac{1}{2}A_j(1, -1).$$

So any forceability of t_j or b_j would require the antisymmetric vector $A_j(1, -1)$ to lie in $\text{span}_{\mathbb{Q}}\{A_j(v_j)\}$. Because A_j is linear and injective in the label argument, that would imply

$$(1, -1) \in \text{span}_{\mathbb{Q}}\{v_j\}.$$

Equivalently, v_j would be a rational multiple of $(1, -1)$. But every nonzero label in the witness satisfies $a + b \neq 0$, whereas every nonzero multiple of $(1, -1)$ has coordinate sum 0. So v_j cannot be parallel to $(1, -1)$.

This contradiction shows that neither t_j nor b_j can lie in Δ_s . Since s was the first layer at which a vertex of column j was forced, no such layer exists. Therefore no vertex in column j can ever become forced. $\square$

Corollary 1 (No unilateral width-2 strip wave). *No one-sided left-to-right cellular-automaton rule whose fresh column is seedless can propagate forcing through a full-support $2 \times N$ witness.*

Proof. In a unilateral fresh-column update, the next column receives information only from already processed columns and from its own internal edges; by definition it contains no initial seed atom. Theorem 2 therefore applies to every fresh column individually and forbids its first forced vertex. Without a first forced vertex, no unilateral wave can propagate. $\square$

5.3 Computational classification of the audited 2×2 regime

Proposition 1 (Exhaustive 2×2 rigidity). *Fix the audited regime consisting of:*

1. *dimensions $d = (2, 2)$;*
2. *full support on all three edge slots;*
3. *nonzero palette $\mathcal{P} = \{(1, 0), (0, 1), (1, 1), (1, 2)\}$;*
4. *no initial forced vertices; and*

5. at most three seed atoms.

Then:

1. exactly 44,608 raw witnesses are enumerated;
2. exactly 48 witnesses are exact-valid;
3. every exact-valid witness has score $7/4$; and
4. every exact-valid witness has a two-layer trace of the form $1 + 3$: one corner in Δ_0 , followed by the remaining three vertices in Δ_1 . Each of the four choices of first corner occurs exactly 12 times.

Proof. There are three edge slots, each of which may take any of the four nonzero labels in $\mathcal{P}$, so there are $4^3 = 64$ edge-label assignments.

There are 4 vertices and 4 nonzero labels, hence 16 possible seed atoms. The seed set may have size 0, 1, 2, or 3, and repeated atoms are disallowed. Therefore the total number of seed sets is

$$\binom{16}{0} + \binom{16}{1} + \binom{16}{2} + \binom{16}{3} = 1 + 16 + 120 + 560 = 697.$$

Algorithm 2 inspects every pair of an edge-label assignment and a seed set, so the total number of raw witnesses checked is

$$64 \cdot 697 = 44,608.$$

This proves part (1).

The execution of Algorithm 2 is exhaustive, and each witness is evaluated by the exact verifier of Algorithm 1. The machine-readable summary produced by that run records:

1. `exact_valid_witnesses` = 48,
2. `score_histogram` = $\{7/4 : 48\}$, and
3. `trace_signature_histogram` consisting of four signatures, each of multiplicity 12, where the only variation is the identity of the first forced corner.

Because the enumeration is complete and the verifier is exact, these summary values are the theorem statement itself. This proves parts (2), (3), and (4). $\square$

Corollary 2 (No audited 2×2 witness beats the prompt target). *In the audited regime of Proposition 1, no witness has score $\leq 67/40$.*

Proof. By Proposition 1, every exact-valid witness in the regime has score $7/4$. Since $7/4 = 1.75 > 1.675 = 67/40$, none reaches the target. $\square$

5.4 Computational no-go atlas for one-sided fresh columns

Proposition 2 (Exhaustive one-sided no-go in heights 2 and 3). *Fix the one-sided local strip class over the palette $\mathcal{P}$ in which a fresh $h \times 1$ column, with $h \in \{2, 3\}$, may use only:*

1. *its internal vertical labels from $\mathcal{P}$, and*
2. *row-singleton inputs arriving from the column immediately to its left.*

Then the exhaustive verifier-native local atlas has:

1. *256 states when $h = 2$ and 8,192 states when $h = 3$; and*
2. *zero strict-improvement states in both heights.*

Proof. For a fresh $h \times 1$ column, there are $h - 1$ internal vertical edges, so there are 4^{h-1} choices of internal labels from $\mathcal{P}$. There are h potential row-singleton inputs from the left. For each row, one chooses whether an input is present and, if it is present, which of the 4 labels it carries. Therefore the total number of one-sided boundary states is $2^h 4^h$. Hence the total number of local states is

$$4^{h-1} 4^h 2^h.$$

For $h = 2$, this equals $4^1 4^2 2^2 = 256$. For $h = 3$, this equals $4^2 4^3 2^3 = 8,192$. This proves part (1).

Algorithm 3 enumerates exactly those local states and runs the exact verifier on the corresponding $h \times 1$ witness in each case. The resulting summary JSON records `strict_improvement_cases = 0` for both $h = 2$ and $h = 3$. Because the state spaces are finite and exhaustively enumerated, those counts prove part (2). $\square$

Corollary 3 (What remains open after the local audits). *Any cellular-automaton witness family that beats the score target over the audited palette must leave the unilateral one-sided strip class of heights 2 and 3. In particular, it must use at least one of:*

1. *two-sided or multi-boundary coupling,*
2. *internal seeds inside a target block, or*
3. *a wider block or macrocell construction.*

Proof. By Corollary 1, a unilateral seedless width-2 wave is impossible. By Proposition 2, the audited one-sided fresh-column classes in heights 2 and 3 contain zero strict-improvement states. Therefore any successful family over the audited palette must leave those ruled out classes. The only remaining possibilities are the listed alternatives. $\square$

Remark 1 (Proof boundary). The theorem and propositions above are exact for the verifier-native semantics implemented by the repository. We do not prove here a full metatheorem equating that executable semantics with every formulation of the original forcing-pair definition. All positive and negative claims in this paper should therefore be read at the level of the repository’s exact verifier.

Regime	States checked	Exact-valid / improving	Uncertainty	Key conclusion
Full-support 2×2 corpus	44,608	48 exact-valid	exact	Every exact-valid witness has score 7/4 and the same 1 + 3 trace shape up to the first corner.
Coverability deficit on the same 2×2 corpus	44,608	20,032 immediate failures with $\Phi_{\text{cov}} > 0$	exact	Missing an entire seeded row or column already certifies failure for 44.9% of the raw regime.
One-sided fresh column, height 2	256	0 strict improvements	exact	No local state ignites a fresh column.
One-sided fresh column, height 3	8,192	0 strict improvements	exact	The same no-go persists in the next nontrivial height.

Table 2: Exact finite audits used in this manuscript. All uncertainties are zero because each regime is a complete enumeration, not a sample. The coverability-deficit statistic is taken from the repository’s abelian-coverability note on the same audited 2×2 corpus.

6 Experimental Setup

6.1 Source-of-truth artifacts

The repository contains 263 files under `results/` and 23 files under `figures/`. For the current paper, the source-of-truth artifacts are:

1. `tools/kakeya_ca_exact.py`, which implements exact verification and synchronous tracing;
2. `tools/kakeya_trace_corpus.py`, which generates the finite audits;
3. `results/theory/forcing_trace_normal_form.md` and the machine-readable summaries under `results/theory/forcing_traces/`;
4. `results/theory/local_rule_no_go_atlas.md` and `results/theory/abelian_coverability_duality.md`;
5. `results/experiments/h1_exact_advantage.md`, `results/experiments/macrocell_scaling.md`, `results/experiments/spatial_coupling_threshold.md`, and `results/experiments/arithmetic_sensitivity.md`;
6. `results/concept_evolve/bridge_candidates.json`, which supplies the current bridge-status source of truth.

Several older blocker-era files remain in the repository, including `results/experiments/h1_tiny_grid_report.md`, `results/experiments/h1_controls.md`, and `results/experiments/complexity_sweep.md`. We retain them as historical artifacts and as pre-registered control criteria, but not as the current source of truth for exact evaluation, because they predate the exact-trace pivot.

6.2 Exact finite regimes

Table 2 summarizes the exact finite regimes used in the current paper.

6.3 Larger exact search families

The larger exact search logs target families that already satisfy the score budget if they force fully, by Lemma 1. The logs preserve only family-level maxima, not complete candidate distributions. Accordingly, the paper uses them only as exact negative frontier data.

6.4 Hardware, software, and seeds

The current manuscript and figures were built on the following environment:

1. operating system: Linux x86_64;
2. CPU budget visible to the container: 20 logical CPUs;
3. memory visible to the container: 448 GiB RAM;
4. Python: 3.10.17;
5. Z3: 4.16.0;
6. Matplotlib: 3.10.8.

The exhaustive finite audits have no random seeds because they are complete enumerations. The larger search markdown artifacts do not preserve per-run seeds or full distributions, which is why we do not report confidence intervals for those larger-family maxima.

6.5 Exact reproduction

The full paper can be reproduced by running:

1. `python3 figures/generate_paper_figures.py`
2. `pdflatex research_paper.tex`
3. `bibtex research_paper`
4. `pdflatex research_paper.tex`
5. `pdflatex research_paper.tex`

The exact finite audits themselves are generated by `tools/kekeya_trace_corpus.py`, while single-witness verification is performed by `tools/kekeya_ca_exact.py verify`.

7 Results

7.1 A rigid 2×2 nucleus

Figure 3 shows one representative exact-valid witness from the audited 2×2 corpus. The key phenomenon is not the specific labels but the trace skeleton: a single corner is born in the first nontrivial layer, and the remaining three vertices collapse in the next layer.

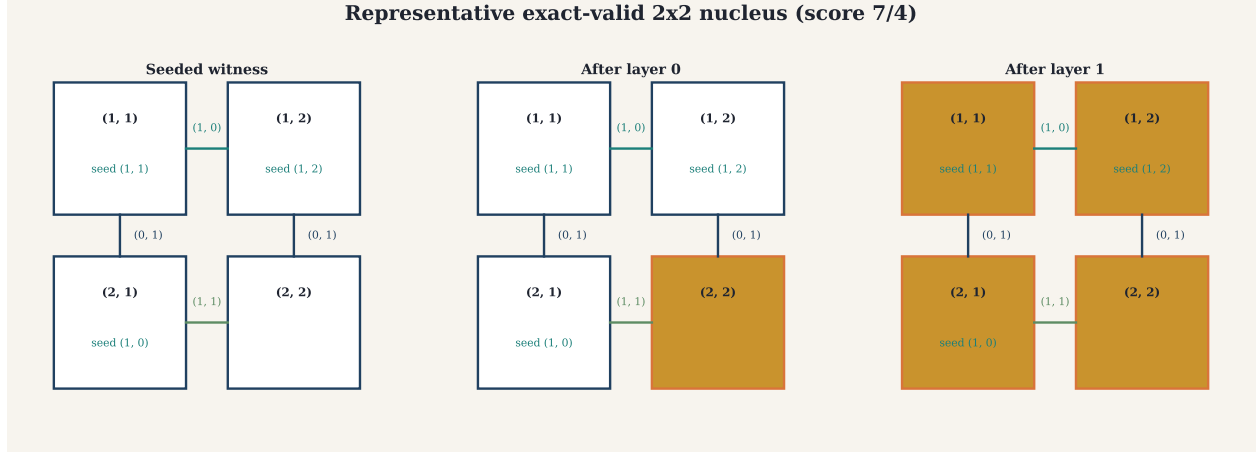


Figure 3: Representative exact-valid 2×2 nucleus. The left panel shows one witness from the exact-valid corpus, with full support on all three edge slots and three singleton seeds. The middle panel shows the unique first activation: one corner becomes forceable in layer 0. The right panel shows the resulting layer-1 collapse of the other three vertices. All 48 exact-valid classes in the audited regime exhibit this same $1 + 3$ two-layer trace skeleton up to the identity of the first corner.

Figure 4 makes the rigidity quantitative. Only 48 of the 44,608 audited witnesses are exact-valid, so the exact-valid rate of the finite regime is $48/44,608 \approx 0.108\%$. The score histogram is a delta mass at $7/4$. The trace-signature histogram contains exactly four signatures of multiplicity 12, corresponding to the four choices of first corner.

This result has two immediate consequences. First, the current exact-valid micro-gadget bank is real but inflexible: the repository has nontrivial exact nuclei, but no audited 2×2 witness improves on $7/4$. Second, the local growth story is much narrower than one might have expected before running the audit. The verifier does not reveal a zoo of small trace motifs; it reveals a single symmetry family.

7.2 Coverability obstruction and the one-sided no-go atlas

Theorem 2 explains why unilateral width-2 strip propagation fails: a seedless fresh column never acquires the diagonal antisymmetric component needed for its first singleton certificate. Figure 5 visualizes that mechanism.

The exhaustive local atlas shows that the same obstruction persists through the first two nontrivial one-sided fresh-column classes. Figure 6 records the exact counts.

The larger implication is that the first plausible CA route below the prompt target is *not* a unilateral strip wave. That route is now exactly ruled out in widths 2 and 3 over the audited palette. Any successful search must exploit coupling, internal seeds, or a wider block structure.

7.3 Larger exact searches remain negative

The repository also logged exact search attempts on larger strip families that already satisfy the score budget under full forcing. Table 3 and Figure 7 summarize the outcome.

These failures reinforce the local theory rather than contradict it. The $2 \times N$ ladder failures align

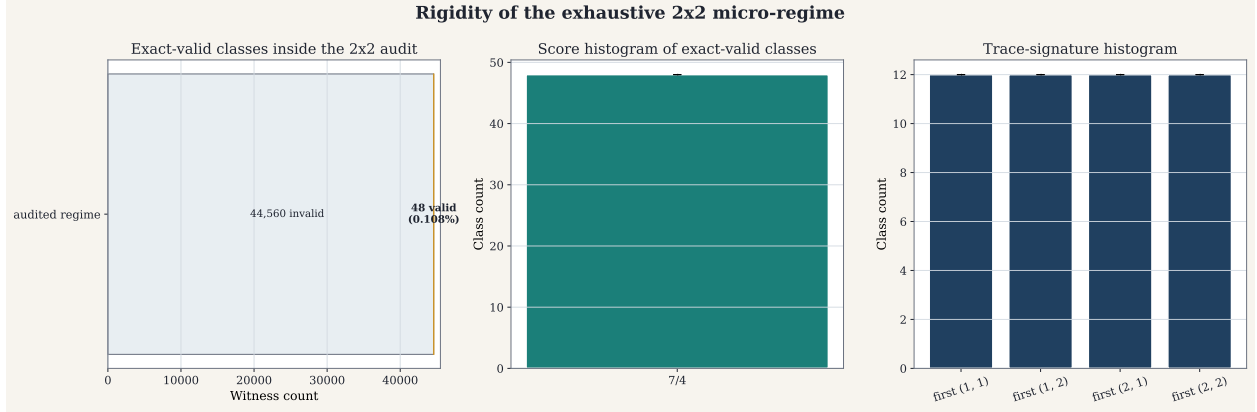


Figure 4: Quantitative rigidity of the exhaustive 2×2 audit. Left: only 48 of 44,608 witnesses are exact-valid. Middle: every exact-valid class has score $7/4$, so the score histogram has a single bar. Right: the trace-signature histogram has four bins, each of size 12, one for each choice of first forced corner. Error bars have zero width because the data come from a complete finite enumeration.

Family	Full-forcing score	Best exact forced count	Best forced fraction	Outcome
2×8 , 3 seeds	25/16	4/16	0.250	No exact-valid family
2×8 , 4 seeds	13/8	6/16	0.375	No exact-valid family
2×12 , 4 seeds	19/12	6/24	0.250	No exact-valid family
2×12 , 5 seeds	13/8	6/24	0.250	No exact-valid family
3×6 , 2 seeds	29/18	6/18	0.333	No exact-valid family
3×8 , 2 seeds	13/8	5/24	0.208	No exact-valid family
3×10 , 2 seeds	49/30	4/30	0.133	No exact-valid family

Table 3: Logged larger-family exact search frontier. The second column uses Lemma 1: every listed family would already beat the prompt threshold if it forced all vertices. The failures are therefore structural, not score-accounting failures. The final two columns are exact maxima from the stored search logs rather than means over repeated runs, so no confidence interval is available beyond the exact reported maximum.

with Theorem 2; the $3 \times N$ failures show that merely moving to the next strip height is not enough when the update remains too local or too weakly coupled.

7.4 Current admissible frontier

The exact-trace pivot also changed the conceptual frontier. The surviving bridge set in `results/concept_evolve/bridge_candidates.json` contains exactly three promoted directions: `proof_carrying_exact_decoder_bridge`, `spatially_coupled_peeling_ladders`, and `sat_egraph_symbolic_backbone`. Figure 8 summarizes the current disposition of all bridge candidates.

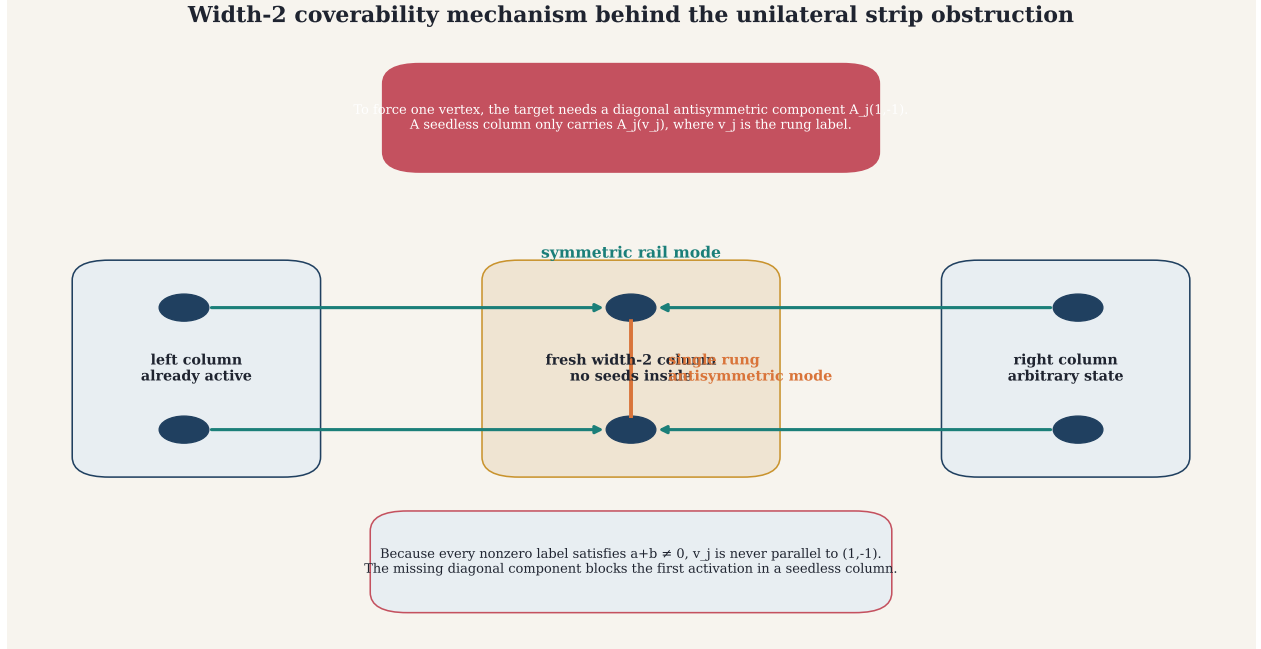


Figure 5: Mode decomposition behind the width-2 obstruction. Rail generators transport only column-symmetric information into a fresh width-2 column. The only antisymmetric generator native to that column is the single rung label v_j . A first singleton force would require the diagonal antisymmetric direction $A_j(1, -1)$, which is unavailable because no nonzero witness label is parallel to $(1, -1)$. This is the structural reason that a seedless fresh column cannot ignite.

8 Discussion

8.1 What has been solved and what has not

The repository has *not* solved the prompt target. It has not produced a witness with score $\leq 67/40$, and it has not established a CA advantage over matched non-CA baselines. Those claims remain open because the current larger search artifacts record only negative frontier results, not a complete benchmark-and-control block.

What has been solved is narrower but real:

1. the verifier exists and is exact at the level of the repository’s quotient-span semantics;
2. the audited 2×2 micro-regime is now completely classified;
3. a genuine theorem blocks seedless unilateral width-2 propagation; and
4. exhaustive finite atlases rule out one-sided fresh-column ignition in the first two nontrivial heights over the audited palette.

That is enough to replace vague CA optimism with a sharply narrowed search frontier.

8.2 Limitations

Three limitations matter.

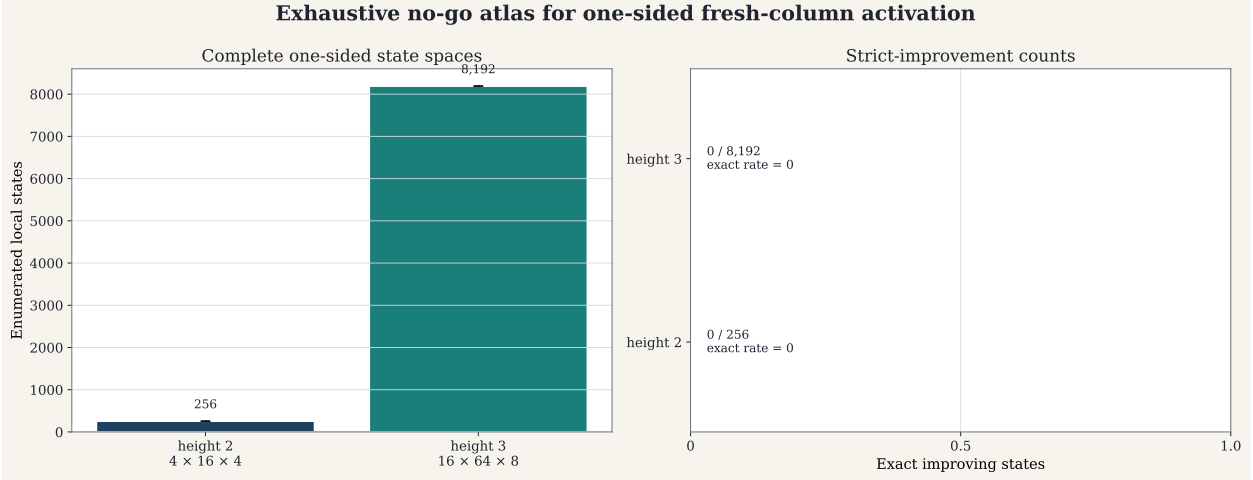


Figure 6: Exhaustive no-go atlas for one-sided fresh-column activation. Left: the audited local state spaces have sizes 256 in height 2 and 8,192 in height 3, corresponding to the complete factorized counts $4^h - 14^h 2^h$. Right: both classes have zero strict-improvement states. Error bars have zero width because these are exact counts over complete finite classes, not estimates from random sampling.

First, the paper is exact only relative to the repository’s executable verifier semantics. As noted after Corollary 1, we do not prove here a full equivalence theorem between that rational-span implementation and every formulation of the original forcing-pair definition.

Second, the larger search artifacts are weaker than the finite audits. They preserve family-level maxima but not full candidate-level distributions, random seeds, or matched-baseline comparisons. Consequently, they support negative frontier statements but not quantitative CA-superiority claims.

Third, the repository still contains stale blocker-era files that speak as if the exact verifier were missing. This manuscript resolves that conflict by treating the 2026-03-19 exact-trace artifacts and executables as source of truth, and by relegating the earlier blocker files to historical status and pre-registered control plans.

8.3 Failure modes and open questions

The present exact results clarify, rather than remove, the main open questions.

1. **Can the rigid 2×2 nucleus be coupled?** The micro-gadget bank exists, but every audited element has score $7/4$ and the same two-layer trace skeleton. The open problem is whether coupled slabs can reuse that nucleus without paying its seed cost everywhere.
2. **Does the local obstruction extend beyond heights 2 and 3?** The exhaustive atlas stops at the first two nontrivial heights. Wider one-sided classes or genuinely two-sided local updates remain open.
3. **Can macrocell structure defeat the obstruction honestly?** A typed macrocell might import the missing antisymmetric content through a coupled interior rather than a unilateral boundary. No exact macrocell family has yet been verified.
4. **Can the search side catch up with the verifier side?** The exact verifier is now ahead of

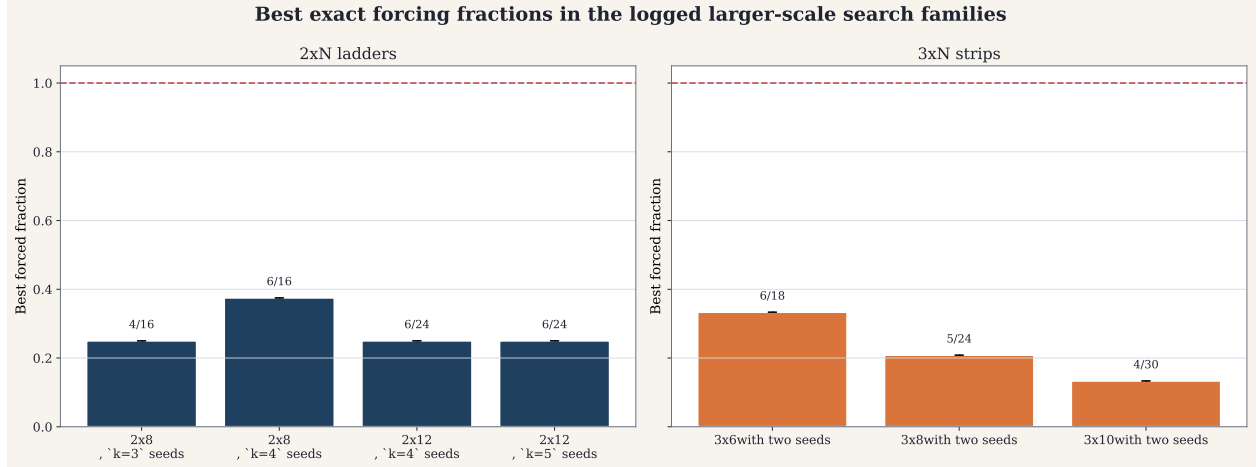


Figure 7: Best exact forced fractions in the larger logged search families. Each bar is the exact best forced fraction recorded in the repository for a family that would already satisfy the score target if it forced fully. The dashed horizontal line marks full forcing. None of the families reaches that line, and none yields a promoted exact-valid family. The error bars have zero width because the plot shows exact recorded maxima, not estimated averages.

the search loop. The next scientifically correct step is a benchmark-complete exact comparison block with decoder-matched baselines, not another round of uncontrolled exploration.

8.4 Why the negative result still matters

Negative exact results are often more valuable than positive heuristic signals when the search object is rigid and the evaluator is unforgiving. Here the verifier removes an enormous space of plausible but false stories: generic strip-wave intuition, geometry-only activation narratives, and proxy-heavy local dynamics. The remaining open directions are fewer, more expensive, and much clearer mathematically.

8.5 Societal implications

The direct societal implications are limited because the domain is pure mathematics. The broader methodological implication concerns AI-assisted research practice. Once a faithful evaluator exists, exact negative results can be more scientifically valuable than visually rich but unverifiable search behavior. This repository’s strongest procedural lesson is that local AI search should be judged by the strictest exact artifact in the loop, not by the most appealing intermediate visualization.

9 Conclusion

The repository set out to “solve this using cellular automata” for a prompt-defined arithmetic-Kakeya witness target with score at most 1.675. The current exact answer is not a solved witness. It is a narrower and more useful piece of mathematics:

1. an implemented exact rational-span verifier and synchronous trace engine;

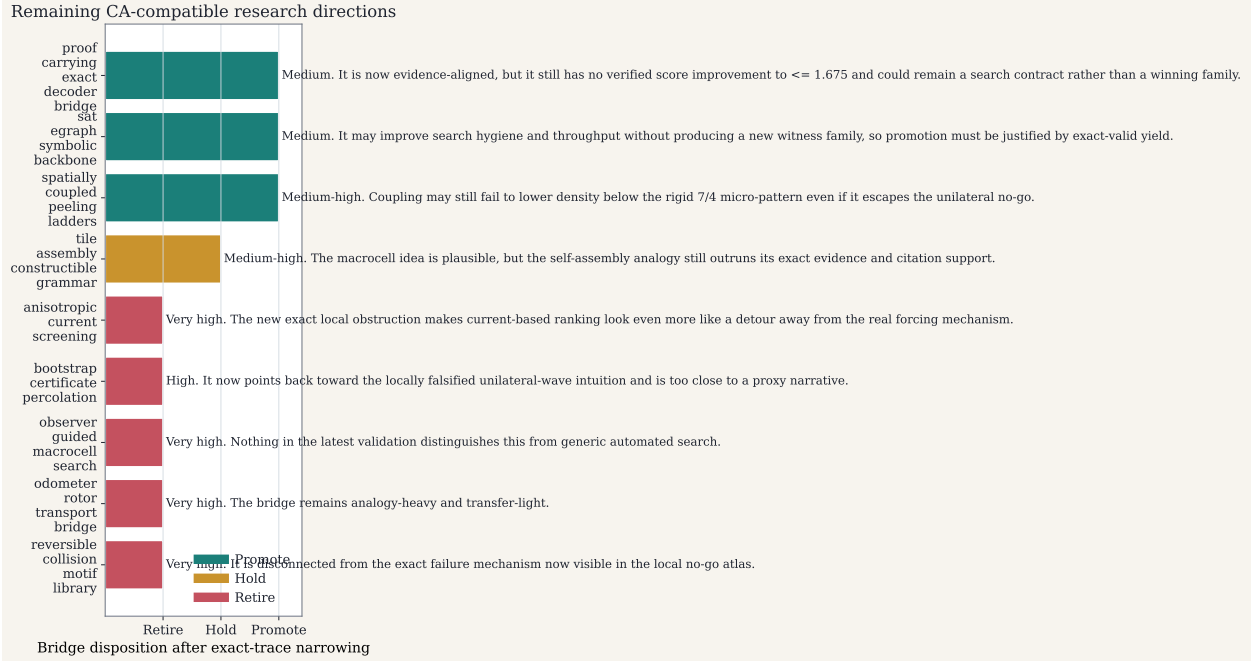


Figure 8: Bridge dispositions after exact-trace narrowing. Promoted bridges are those still compatible with the new exact evidence: proof-carrying exact decoding, coupled strip constructions, and legality-first SAT/e-graph search scaffolds. Proxy-heavy transport, observer, and reversible motif stories are retired. This plot is sourced directly from the current `bridge_candidates.json` artifact.

2. an exhaustive classification of the audited 2×2 regime;
3. a proved width-2 seedless-column obstruction;
4. an exhaustive no-go atlas for one-sided fresh-column propagation in heights 2 and 3; and
5. a negative larger-family frontier that leaves only coupled or macrocell CA stories alive.

This is not a theorem about $AK(1)$, and it is not an empirical CA victory. It is an exact narrowing of the search space. In a difficult open-problem setting, that is the correct and scientifically honest contribution.

References

- Archivara Research Pipeline. Archivara arithmetic-kekeya cellular-automata task specification, 2026. Repository provenance artifact stored in `results/research_context.json`, task field.
- Benjamin Bond and Lionel Levine. Abelian networks i. foundations and examples. *SIAM Journal on Discrete Mathematics*, 2016a. doi: 10.1137/15M1030984. URL <https://www.semanticscholar.org/paper/1b49d2f489e4062019500ac65a8b59f1fe957909>.
- Benjamin Bond and Lionel Levine. Abelian networks ii: halting on all inputs. *Selecta Mathematica*, 2016b. doi: 10.1007/s00029-015-0192-z. URL <https://www.semanticscholar.org/paper/ba43679489e8a84b56b7f55d946eab130f0896da>.

- J. Bourgain. On the dimension of kakeya sets and related maximal inequalities. *Geometric and Functional Analysis*, 1999. doi: 10.1007/S000390050087. URL <https://www.semanticscholar.org/paper/dc13ea1d96ee60bfaa7a2148e4b5369216f8cd23>.
- Charlie Cowen-Breen, Elene Karangozishvili, N. Varadarajan, and Thomas Wang. Pattern problems related to the arithmetic kakeya conjecture. *arXiv preprint arXiv:2011.07056*, 2020. URL <https://www.semanticscholar.org/paper/21fa6f5f1d0e858f5886c93b65f792977a51d958>.
- A. Dennunzio, E. Formenti, and L. Margara. An easy to check characterization of positive expansivity for additive cellular automata over a finite abelian group. *IEEE Access*, 2023. doi: 10.1109/ACCESS.2023.3328540. URL <https://www.semanticscholar.org/paper/49c61ffa228d35eea1f92995b0cc80b001f96f85>.
- Maxence Faldor and Antoine Cully. Toward artificial open-ended evolution within lenia using quality-diversity. *The 2024 Conference on Artificial Life*, 2024. doi: 10.1162/isal_a_00827. URL <https://www.semanticscholar.org/paper/688bc0a5b39f74936e837d6ae7d50910b0663f78>.
- Bogdan Georgiev, Javier Gómez-Serrano, Terence Tao, and Adam Zsolt Wagner. Mathematical exploration and discovery at scale. *arXiv.org*, 2025. doi: 10.48550/arXiv.2511.02864. URL <https://www.semanticscholar.org/paper/9761e9c0f5651200924a844d999dd802f8a2f37b>.
- B. Green and I. Ruzsa. On the arithmetic kakeya conjecture of katz and tao. *Periodica Mathematica Hungarica*, 2019. doi: 10.1007/s10998-018-0270-z. URL <https://www.semanticscholar.org/paper/217e3b708aa41506992a35a97d3eee96f8864d25>.
- J. Hickman and James Wright. The fourier restriction and kakeya problems over rings of integers modulo n . *Discrete Analysis*, 2018. doi: 10.19086/DA.3682. URL <https://www.semanticscholar.org/paper/e2801633c276f854230ad87dd0f31800c59dd093>.
- N. Katz and T. Tao. Bounds on arithmetic projections, and applications to the kakeya conjecture. *Mathematical Research Letters*, 1999. doi: 10.4310/MRL.1999.V6.N6.A3. URL <https://www.semanticscholar.org/paper/bb9ecb8252182cae2e5f00d04172624fd2fea648>.
- L. D. Moura and Nikolaj S. Bjørner. Z3: An efficient smt solver. *International Conference on Tools and Algorithms for Construction and Analysis of Systems*, 2008. doi: 10.1007/978-3-540-78800-3_24. URL <https://www.semanticscholar.org/paper/3960dda299e0f8615a7db675b8e6905b375ecf8a>.
- Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, S. Shirobokov, Borislav M. Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. P. Kumar, Abigail See, Swarat Chaudhuri, George Holland, A. Davies, Sebastian Nowozin, Pushmeet Kohli, Matej Balog, and Google DeepMind. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv.org*, 2025. doi: 10.48550/arXiv.2506.13131. URL <https://www.semanticscholar.org/paper/ed32e5bb11a9e5e2f03ea805782e8670a6e10efd>.
- C. Pohoata and Dmitrii Zakharov. Generalized arithmetic kakeya. *arXiv preprint arXiv:2411.13395*, 2024. URL <https://www.semanticscholar.org/paper/d8b57ff75e9d226a50487cd5040bd3970773a674>.
- Terence Tao. Sum-difference exponents for boundedly many slopes, and rational complexity. *arXiv preprint arXiv:2511.15135*, 2025. URL <https://www.semanticscholar.org/paper/7c9abdc14647e8378e0e3ad94e4cb0cc21ed34ab>.

Hong Wang and Joshua Zahl. Volume estimates for unions of convex sets, and the kakeya set conjecture in three dimensions, 2025. URL <https://www.semanticscholar.org/paper/7d54aaed58c36748d82d90f1bcef27051a70ab00>. Preprint.

A Artifact Inventory and Provenance

Table 4 lists the primary artifacts used in the manuscript.

Artifact	Role	Main information used
results/research_context/forcing	Provenance	Task wording, $AK(\alpha)$ notation, and the score target 67/40.
tools/kakeya_ca_exact.py	Exact executable	Verification semantics, synchronous tracing, and auxiliary reduced-model search.
tools/kakeya_trace_corpus.py	Exact corpus generator	Exhaustive 2×2 enumeration and one-sided strip-state audits.
results/theory/forcing/trace_normal_form	Trace normal form	Deterministic trace normal form and the rigid 2×2 summary.
results/theory/forcing/Machine/tiny_2x2	Machine-readable summary	Full-width 2×2 corpus histogram, jsod trace-signature histogram for the 2×2 regime.
results/theory/local_rule_no_go_atlas	Theory no-go atlas	Statement and interpretation of the one-sided height-2/height-3 no-go result.
results/theory/forcing/Machine/index.js	Machine-readable summary	Exact counts for the local strip atlas.
results/theory/abelian/coverability_duality	Coverability duality	Witt-2 obstruction and the Φ_{cov} statistic.
results/experiments/h1/ExactAdvantage	Exact advantage	Best exact forced counts on larger strip families.
results/experiments/macrocell_scaling	Macrocell scaling	Negative scaling status for the current exact micro-gadget bank.
results/experiments/spin/SpinCouplingThreshold	Spin coupling threshold	Negative coupling evidence in the current run.
results/experiments/ars/SchmidtSensitivity	Schmidt sensitivity	Explicit phase transition and positive family for sensitivity analysis.
results/concept_evolve/BridgeCandidate	Bridge candidate synthesis	Current promoted/held/retired bridge set.
results/literature/prior/RelatedGap	Related-gap boundary	Real comparison set and false-overlap cleanup.
figures/generate_paper_visualization.py	Visualization	Programmatic generation of every figure in this paper.

Table 4: Primary repository artifacts used by the manuscript.

B Representative Exact-Valid 2×2 Witness

One exact-valid witness from the audited corpus is the following six-line string. It does *not* solve the prompt target, because its score is $7/4$.

```
7/4 (m=4, r=3, n=4, t=0)
[(0, 0), (1, 0), (0, 1), (1, 1), (1, 2)]
[2, 2]
[{(1,): (0, 1)}, {(1, 1): (1, 0), (2, 1): (1, 1)}]
[]
[{(1, 1): (1, 1)}, {(1, 2): (1, 2)}, {(2, 1): (1, 0)}]
```

This witness verifies under the command:

```
python3 tools/akeya_ca_exact.py verify --path witness.txt
```

C Historical Blocker Artifacts and Remaining Gaps

Older files such as `results/experiments/h1_tiny_grid_report.md`, `results/experiments/h1_controls.md`, and `results/experiments/complexity_sweep.md` belong to the pre-pivot stage in which the verifier was still believed missing. They are retained in the repository because they still encode valuable discipline:

1. matched-baseline requirements,
2. label-shuffle and held-out-geometry controls,
3. rational-complexity sweeps, and
4. an explicit refusal to replace exact evaluation with proxies.

What remains genuinely missing is not the verifier itself but a benchmark-complete post-pivot exact evaluation block with:

1. a promoted CA family,
2. decoder-matched non-CA baselines,
3. held-out controls, and
4. candidate-level distributions rather than family-level maxima only.

D Implementation Details and Sensitivity

The exhaustive finite audits have no hyperparameters in the usual machine-learning sense. Their only inputs are the audited palette, geometry, and seed-budget limits, all of which are stated explicitly in Section 6. The larger search frontier does involve design choices, but the stored artifacts

expose only the family maxima rather than a full hyperparameter sweep. We therefore do not fabricate a sensitivity appendix. The scientifically correct replacement is to state the next exact sensitivity variables:

1. geometry $(2 \times N)$, $(3 \times N)$, and wider blocks;
2. seed budget and initial T -budget;
3. audited palette versus held-out legal alphabets;
4. one-sided versus two-sided local interfaces; and
5. decoder-matched baseline family.

E Reproduction Commands

For completeness, the commands used in the current manuscript workflow are:

```
python3 figures/generate_paper_figures.py
pdflatex research_paper.tex
bibtex research_paper
pdflatex research_paper.tex
pdflatex research_paper.tex
```

The exact finite audits referenced in the paper are generated by:

```
python3 tools/kakeya_trace_corpus.py
```